LAPORAN UTS MACHINE LEARNING

KLASIFIKASI PENYAKIT DIABETES MENGGUNAKAN ALGORITMA DECISION TREE DENGAN PENANGANAN DATA TIDAK SEIMBANG (SMOTE)



Kelompok 2

Disusun Oleh :

Ziyaa Danil M	434241034
M. Ainul Yaqin	434241036
Vicky Candra Arifian	434241038

PROGRAM STUDI D4 TEKNIK INFORMATIKA

UNIVERSITAS AIRLANGGA

SURABAYA

2026

DAFTAR ISI

BAB I PENDAHULUAN..... 5
1.1 Latar Belakang..... 5
1.2 Rumusan Masalah..... 6
1.3 Tujuan..... 6
BAB II LANDASAN TEORI..... 7
2.1 Data Mining dan Klasifikasi..... 7
2.2 Algoritma Decision Tree..... 7
2.3 Pra-pemrosesan Data (Preprocessing)..... 7
2.4 Transformasi Data (MinMaxScaler)..... 8
2.5 Data Tidak Seimbang dan SMOTE..... 8
2.6 Evaluasi: Confusion Matrix dan Metrik..... 8
BAB III METODOLOGI..... 10
3.1 Dataset..... 10
3.2 Alat dan Bahan..... 10
3.3 Alur Laporan..... 10
BAB IV IMPLEMENTASI DAN PEMBAHASAN..... 11
4.1 Import Library..... 11
4.2 Input Data..... 11
4.3 Preprocessing..... 12
4.4 Split Data..... 13
4.5 Transformasi (MinMaxScaler)..... 14
4.6 Resampling (Oversampling dengan SMOTE)..... 14
4.7 Training (Decision Tree)..... 16
4.8 Testing..... 17
4.9 Evaluasi..... 17
BAB V PENUTUP..... 19
5.1 Kesimpulan..... 19
5.2 Saran..... 19
DAFTAR PUSTAKA..... 20

DAFTAR GAMBAR

Gambar 3.1 Flowchart alur program klasifikasi 9
Gambar 4.1 Kode import library yang dibutuhkan 10
Gambar 4.2 Kode tahap Input Data 10
Gambar 4.3 Output tahap Input Data 11
Gambar 4.4 Kode tahap Preprocessing 11
Gambar 4.5 Output tahap Preprocessing 12
Gambar 4.6 Kode tahap Split Data 12
Gambar 4.7 Output tahap Split Data 13
Gambar 4.8 Kode tahap Transformasi 13
Gambar 4.9 Grafik distribusi kelas SEBELUM SMOTE (imbalance) 14
Gambar 4.10 Grafik distribusi kelas SESUDAH SMOTE (seimbang) 15
Gambar 4.11 Kode tahap Training 15
Gambar 4.13 Kode tahap Testing 16
Gambar 4.14 Kode tahap Evaluasi 16
Gambar 4.15 Output kode Evaluasi 17

BAB I PENDAHULUAN

1.1 Latar Belakang

Diabetes melitus merupakan suatu gangguan metabolisme yang ditandai dengan hiperglikemia yang diakibatkan oleh ketidakmampuan tubuh untuk mensekresi dan merespons insulin (Olisah et al., 2022). Diabetes melitus menjadi salah satu penyakit kronis penyebab kematian terbesar kelima di seluruh dunia dengan total 1,6 juta kematian (Olisah et al., 2022). Kondisi serupa juga terjadi di Indonesia. Berdasarkan data International Diabetes Federation (2023), prevalensi diabetes pada populasi dewasa di Indonesia mencapai 11,3%, dengan total kasus mencapai 20.426.400 dari 185.217.400 populasi dewasa. Angka ini menempatkan Indonesia sebagai salah satu negara dengan beban diabetes tertinggi di dunia. Oleh karena itu, deteksi dini terhadap risiko diabetes sangat penting agar penanganan dapat dilakukan lebih cepat dan komplikasi dapat dicegah.

Seiring berkembangnya teknologi, data indikator kesehatan masyarakat dalam jumlah besar dapat dimanfaatkan untuk membangun model prediksi menggunakan teknik data mining, khususnya klasifikasi. Pendekatan konvensional berbasis aturan medis sering kali kurang mampu menangani data kesehatan yang kompleks dan berdimensi tinggi, sehingga diperlukan pendekatan berbasis data.

Salah satu tantangan utama pada data kesehatan adalah kondisi data yang tidak seimbang (imbalanced), yaitu jumlah data pada kelas tertentu jauh lebih sedikit dibanding kelas lainnya. Kondisi ini dapat menyebabkan model cenderung memihak kelas mayoritas, sehingga metrik akurasi menjadi tinggi namun recall pada kelas minoritas rendah. Hal ini berimplikasi pada gagalnya deteksi dini pasien diabetes. Untuk mengatasi permasalahan tersebut, laporan ini menerapkan teknik penyeimbangan data Synthetic Minority Over-sampling Technique (SMOTE) yang dikombinasikan dengan algoritma Decision Tree. Decision Tree dipilih karena interpretabilitasnya yang tinggi, sedangkan SMOTE terbukti efektif dalam meningkatkan performa model pada data yang tidak seimbang. Laporan ini

bertujuan untuk membangun dan mengevaluasi model klasifikasi status diabetes yang lebih seimbang dan akurat.

1.2 Rumusan Masalah

- Bagaimana tahapan pra-pemrosesan data yang diperlukan sebelum data digunakan untuk klasifikasi?
- Bagaimana cara menangani data yang tidak seimbang pada dataset diabetes?
- Bagaimana performa algoritma Decision Tree dalam mengklasifikasikan status diabetes?

1.3 Tujuan

- Membangun model klasifikasi status diabetes menggunakan algoritma Decision Tree.
- Menerapkan pra-pemrosesan data, transformasi (MinMaxScaler), dan penyeimbangan data (SMOTE).
- Mengevaluasi performa model menggunakan Confusion Matrix, Akurasi, Presisi, Recall, dan F1-Score.

BAB II LANDASAN TEORI

2.1 Data Mining dan Klasifikasi

Data mining adalah proses menggali informasi atau pola yang bermanfaat dari sekumpulan data berukuran besar. Data mining telah banyak diterapkan di sektor kesehatan dan medis, termasuk untuk perencanaan dan manajemen keperawatan (Attari et al., 2026). Salah satu tugas utama dalam data mining adalah klasifikasi, yaitu proses membangun model untuk memprediksi kelas atau kategori dari suatu data berdasarkan fitur-fiturnya. Klasifikasi termasuk dalam pembelajaran terarah (supervised learning) karena model dilatih menggunakan data yang telah memiliki label.

2.2 Algoritma Decision Tree

Decision Tree (pohon keputusan) adalah algoritma klasifikasi yang berbentuk struktur pohon, terdiri atas simpul akar (root node), simpul cabang (internal node), dan simpul daun (leaf node). Setiap internal node merepresentasikan pengujian terhadap suatu fitur, cabang merepresentasikan hasil pengujian, dan leaf node merepresentasikan kelas akhir (Sharma & Kumar, 2026).

Dalam membangun pohon, algoritma memilih fitur pemisah terbaik berdasarkan ukuran ketidakmurnian (impurity), seperti Entropy dan Information Gain atau indeks Gini. Semakin murni pembagian yang dihasilkan sebuah fitur, semakin baik fitur tersebut dijadikan pemisah. Nilai Entropy dan Gini dihitung dengan rumus berikut:

$$Entropy(S) = -\Sigma\, p_i \log_2 p_i$$

$$Gini(S) = 1 - \Sigma\, (p_i)^2$$

dengan p_i adalah proporsi data yang termasuk kelas ke-i. Kelebihan Decision Tree adalah mudah dipahami dan diinterpretasikan, sedangkan kekurangannya adalah rentan terhadap overfitting terutama bila pohon tumbuh terlalu dalam.

2.3 Pra-pemrosesan Data (Preprocessing)

Pra-pemrosesan adalah tahap membersihkan dan menyiapkan data agar berkualitas sebelum digunakan. Pada laporan ini, tahap ini mencakup tiga hal:

- Missing value: nilai kosong pada data yang dapat mengganggu proses pelatihan model.
- Duplikasi data: baris data yang tercatat lebih dari satu kali sehingga perlu dihapus agar tidak membebani dan membiaskan model.

- Outlier: nilai ekstrem yang menyimpang jauh dari sebaran data. Outlier dideteksi menggunakan metode IQR (Interquartile Range) dengan batas bawah dan batas atas sebagai berikut:

$$IQR = Q3 - Q1$$

$$Batas\ Bawah = Q1 - 1{,}5 \times IQR \quad ; \quad Batas\ Atas = Q3 + 1{,}5 \times IQR$$

2.4 Transformasi Data (MinMaxScaler)

Transformasi data dilakukan untuk menyeragamkan skala antar-fitur. MinMaxScaler adalah teknik normalisasi yang mengubah nilai setiap fitur ke dalam rentang 0 sampai 1 menggunakan rumus:

$$X' = (X - X□_{i}□) / (X□_{ax} - X□_{i}□)$$

Normalisasi penting agar fitur dengan rentang nilai besar (misalnya BMI) tidak mendominasi fitur dengan rentang kecil, sehingga model dapat belajar secara lebih adil. Namun, MinMaxScaler tidak menghilangkan pengaruh outlier karena nilai ekstrem tetap memengaruhi nilai minimum dan maksimum yang digunakan dalam penskalaan (scikit-learn developers, 2026).

2.5 Data Tidak Seimbang dan SMOTE

Data tidak seimbang (imbalanced) terjadi ketika distribusi jumlah data antar-kelas sangat timpang. Model yang dilatih pada data semacam ini cenderung memihak kelas mayoritas dan buruk dalam mengenali kelas minoritas. SMOTE menangani masalah ini dengan membuat sampel sintetis kelas minoritas melalui interpolasi antara sampel minoritas dan tetangga terdekatnya, bukan sekadar menduplikasi data (Chawla et al., 2002).

Berbeda dengan penggandaan biasa, SMOTE membuat data sintetis baru pada kelas minoritas dengan cara melakukan interpolasi di antara data minoritas yang berdekatan (tetangga terdekat). Dengan demikian jumlah data setiap kelas menjadi seimbang tanpa sekadar menyalin data yang sama. SMOTE hanya diterapkan pada data latih untuk menghindari kebocoran data (data leakage).

2.6 Evaluasi: Confusion Matrix dan Metrik

Confusion Matrix adalah tabel yang membandingkan hasil prediksi model dengan nilai sebenarnya. Dari tabel ini diperoleh empat komponen dasar: True Positive (TP), True Negative (TN), False Positive (FP), dan False Negative (FN). Berdasarkan komponen tersebut dihitung metrik evaluasi berikut:

$$Akurasi = (TP + TN) / (TP + TN + FP + FN)$$

$$Presisi = TP / (TP + FP)$$

$$Recall = TP / (TP + FN)$$

$$F1\text{-}Score = 2 \times (Presisi \times Recall) / (Presisi + Recall)$$

Akurasi menunjukkan proporsi prediksi benar secara keseluruhan; presisi menunjukkan ketepatan prediksi positif; recall menunjukkan kemampuan menemukan seluruh data positif; dan F1-Score adalah rata-rata harmonik antara presisi dan recall. Pada data tidak seimbang, accuracy dapat memberikan penilaian yang terlalu optimistis karena kelas mayoritas mendominasi; karena itu, F1-score lebih sesuai digunakan sebagai metrik utama (Chicco & Jurman, 2025; Olisah et al., 2022).

BAB III METODOLOGI

3.1 Dataset

Dataset yang digunakan adalah Diabetes Health Indicators Dataset (BRFSS 2015) yang bersumber dari survei CDC Amerika Serikat. Dataset ini memiliki 253.680 baris dan 22 kolom, terdiri atas 21 fitur indikator kesehatan dan 1 kolom target Diabetes_012. Kolom target memiliki tiga kelas, yaitu kelas 0 (tidak diabetes), kelas 1 (pradiabetes), dan kelas 2 (diabetes).

Distribusi awal kelas target ditunjukkan pada tabel berikut.

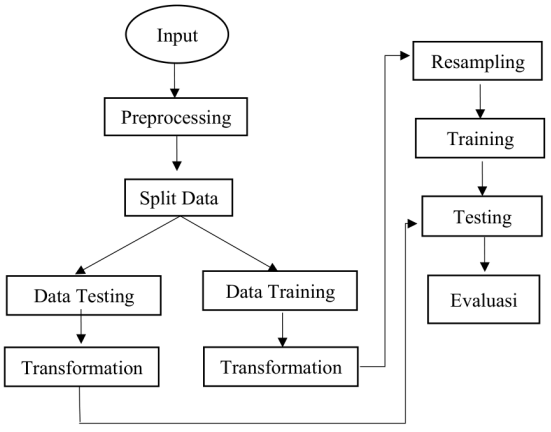
Kelas	Jumlah Data	Persentase
0 (Tidak Diabetes)	213.703	84,24%
1 (Pradiabetes)	4.631	1,83%
2 (Diabetes)	35.346	13,93%

3.2 Alat dan Bahan

Program dibuat menggunakan bahasa pemrograman Python dengan bantuan pustaka pandas dan numpy untuk pengolahan data, scikit-learn untuk pemodelan dan evaluasi, imbalanced-learn (imblearn) untuk SMOTE, serta matplotlib dan seaborn untuk visualisasi.

3.3 Alur Laporan

Program disusun mengikuti alur kerja: Input Data → Preprocessing → Split Data (80:20) → Transformasi (MinMaxScaler) → Resampling/Oversampling (SMOTE) → Training (Decision Tree) → Testing → Evaluasi. Transformasi dan resampling hanya diterapkan pada data latih guna mencegah kebocoran data.



Gambar 3.1 Flowchart alur program klasifikasi

BAB IV IMPLEMENTASI DAN PEMBAHASAN

4.1 Import Library

Tahap pertama adalah mengimpor seluruh *library* yang dibutuhkan. Potongan kode berikut memuat pustaka untuk pengolahan data, pembagian data, transformasi, algoritma Decision Tree, metrik evaluasi, SMOTE, serta visualisasi.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (confusion_matrix, accuracy_score,
                             precision_score, recall_score, f1_score,
                             classification_report)
from imblearn.over_sampling import SMOTE
```

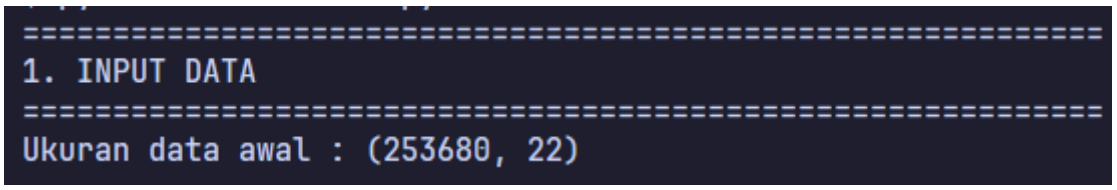
Gambar 4.1 Kode import library yang dibutuhkan

4.2 Input Data

Dataset dibaca menggunakan fungsi read_csv dari pandas, kemudian ditampilkan ukuran datanya untuk memastikan data berhasil dimuat. Hasilnya menunjukkan data berukuran 253.680 baris dan 22 kolom.

```
FILE = "diabetes_012_health_indicators_BRFSS2015.csv"
df = pd.read_csv(FILE)
print("=" * 60)
print("1. INPUT DATA")
print("=" * 60)
print("Ukuran data awal :", df.shape)
```

Gambar 4.2 Kode tahap Input Data



Gambar 4.3 Output tahap Input Data

4.3 Preprocessing

Tahap pra-pemrosesan terdiri dari deteksi missing value, penghapusan duplikasi, dan penanganan outlier menggunakan metode IQR. Nilai yang berada di luar batas IQR ditangani dengan teknik capping, yaitu dipotong ke batas terdekat sehingga tidak mengganggu proses pelatihan.

```
# a) Deteksi missing value
print("\n[a] Missing value per kolom (total):", df.isnull().sum().sum())

# b) Deteksi & hapus duplikasi data
dup = df.duplicated().sum()
print(f"[b] Jumlah data duplikat : {dup}")
df = df.drop_duplicates().reset_index(drop=True)
print("    Ukuran setelah hapus duplikat :", df.shape)

# c) Deteksi & tangani outlier (metode IQR pada fitur numerik kontinu)
kolom_numerik = ["BMI", "MentHlth", "PhysHlth"]
print("[c] Deteksi outlier (IQR) pada:", kolom_numerik)
for kol in kolom_numerik:
    Q1 = df[kol].quantile(0.25)
    Q3 = df[kol].quantile(0.75)
    IQR = Q3 - Q1
    batas_bawah = Q1 - 1.5 * IQR
    batas_atas = Q3 + 1.5 * IQR
    jml_outlier = ((df[kol] < batas_bawah) | (df[kol] > batas_atas)).sum()
    print(f"    {kol:10s}: {jml_outlier} outlier  (batas {batas_bawah:.1f} - {batas_atas:.1f})")
    # Capping: nilai di luar batas dipotong ke batas terdekat
    df[kol] = df[kol].clip(lower=batas_bawah, upper=batas_atas)
print("    Outlier ditangani dengan capping (clip ke batas IQR).")
```

Gambar 4.4 Kode tahap Preprocessing

Hasil pemeriksaan menunjukkan tidak terdapat missing value (0), ditemukan 23.899 baris duplikat yang kemudian dihapus sehingga data tersisa 229.781 baris, serta outlier pada fitur BMI, MentHlth, dan PhysHlth yang telah ditangani.

```
============================================================
2. PREPROCESSING
============================================================

[a] Missing value per kolom (total): 0
[b] Jumlah data duplikat : 23899
    Ukuran setelah hapus duplikat : (229781, 22)
[c] Deteksi outlier (IQR) pada: ['BMI', 'MentHlth', 'PhysHlth']
    BMI       : 5638 outlier  (batas 12.0 - 44.0)
    MentHlth  : 36163 outlier  (batas -3.0 - 5.0)
    PhysHlth  : 34347 outlier  (batas -6.0 - 10.0)
    Outlier ditangani dengan capping (clip ke batas IQR).
```

Gambar 4.5 Output tahap Preprocessing

4.4 Split Data

Data dibagi menjadi 80% data latih dan 20% data uji menggunakan train_test_split. Parameter stratify=y menjaga proporsi tiap kelas tetap sama pada kedua bagian, sedangkan random_state=42 memastikan hasil pembagian dapat direproduksi.

```python
# Pisahkan fitur (X) dan target (y)
X = df.drop(columns=["Diabetes_012"])
y = df["Diabetes_012"]

print("\n" + "=" * 60)
print("3. SPLIT DATA (80% training : 20% testing)")
print("=" * 60)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
print("Data training :", X_train.shape[0], "baris")
print("Data testing  :", X_test.shape[0], "baris")
```

Gambar 4.6 Kode tahap Split Data

Hasil pembagian menghasilkan 183.824 baris data latih dan 45.957 baris data uji.

```
============================================================
3. SPLIT DATA (80% training : 20% testing)
============================================================
Data training : 183824 baris
Data testing  : 45957 baris
```

Gambar 4.7 Output tahap Split Data

4.5 Transformasi (MinMaxScaler)

Seluruh fitur dinormalisasi ke rentang 0–1 menggunakan MinMaxScaler. Objek scaler di-fit hanya pada data latih, lalu digunakan untuk mentransformasi data latih maupun data uji. Cara ini menjaga agar informasi dari data uji tidak ikut memengaruhi proses transformasi.

```python
scaler = MinMaxScaler()
X_train = scaler.fit_transform(X_train)   # fit HANYA pada data training
X_test = scaler.transform(X_test)         # test hanya di-transform
print("Semua fitur diskalakan ke rentang [0, 1].")
```

```
============================================================
4. TRANSFORMASI (MinMaxScaler)
============================================================
Semua fitur diskalakan ke rentang [0, 1].
```

Gambar 4.8 Kode tahap Transformasi

4.6 Resampling (Oversampling dengan SMOTE)

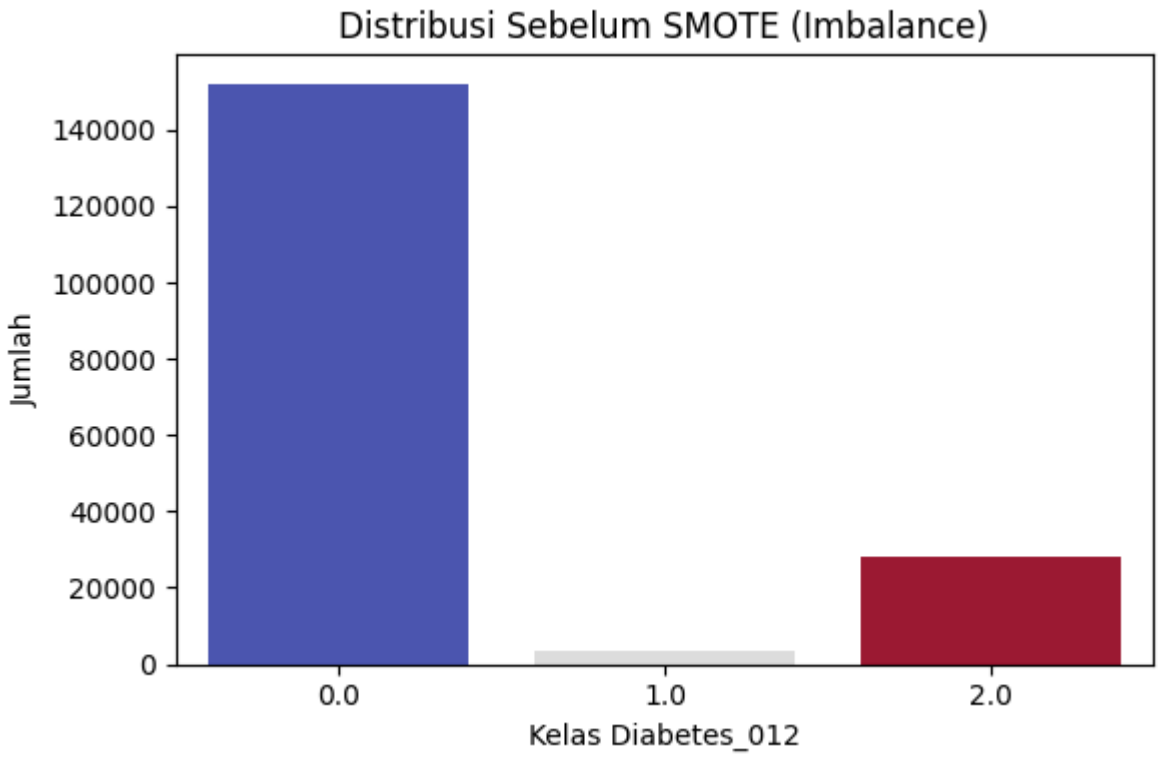
Karena data sangat tidak seimbang, dilakukan oversampling SMOTE pada data latih. Distribusi kelas divisualisasikan dalam bentuk diagram batang sebelum dan sesudah SMOTE untuk memperlihatkan perubahannya.

```
print("\n" + "=" * 60)
print("5. RESAMPLING (Oversampling - SMOTE)")
print("=" * 60)
print("Distribusi kelas SEBELUM oversampling:")
print(pd.Series(y_train).value_counts().sort_index().to_string())

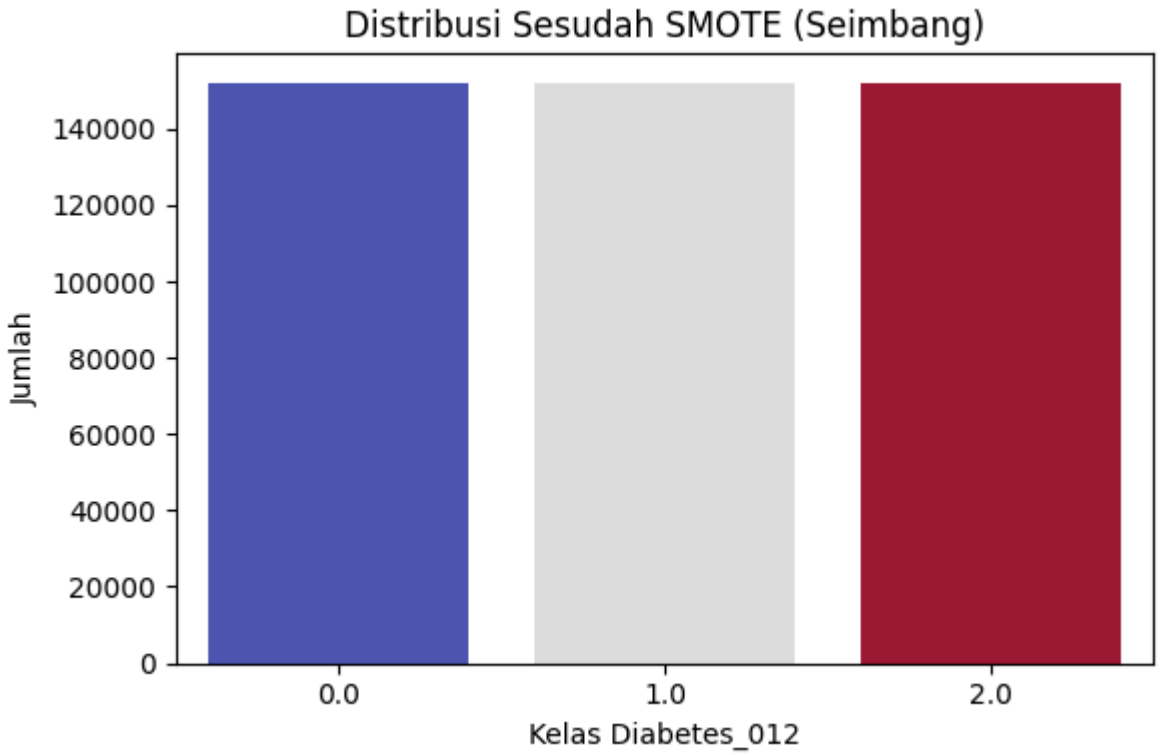
smote = SMOTE(random_state=42)
X_train, y_train = smote.fit_resample(X_train, y_train)

print("\nDistribusi kelas SESUDAH oversampling:")
print(pd.Series(y_train).value_counts().sort_index().to_string())
```

Sebelum SMOTE, jumlah data kelas 0, 1, dan 2 berturut-turut adalah 152.043; 3.703; dan 28.078. Setelah SMOTE, ketiga kelas menjadi seimbang dengan jumlah masing-masing 152.043 data. Perubahan ini terlihat jelas pada kedua grafik berikut.



Gambar 4.9 Grafik distribusi kelas SEBELUM SMOTE (imbalance)



Gambar 4.10 Grafik distribusi kelas SESUDAH SMOTE (seimbang)

4.7 Training (Decision Tree)

Model Decision Tree Classifier dilatih menggunakan data latih yang telah diseimbangkan. Model akan membentuk aturan-aturan keputusan bercabang berdasarkan nilai fitur untuk memisahkan ketiga kelas.

```
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)
print("Model Decision Tree selesai dilatih.")
```

Gambar 4.11 Kode tahap Training

```
============================================================
6. TRAINING (Decision Tree)
============================================================
Model Decision Tree selesai dilatih.
```

Gambar 4.12 Output tahap Training

4.8 Testing

Model yang telah dilatih digunakan untuk memprediksi kelas pada data uji yang belum pernah dilihat sebelumnya. Hasil prediksi ini menjadi dasar penilaian pada tahap evaluasi.

```
y_pred = model.predict(X_test)
```

Gambar 4.13 Kode tahap Testing

4.9 Evaluasi

Model dievaluasi menggunakan Confusion Matrix serta metrik Akurasi, Presisi, Recall, dan F1-Score dengan rata-rata weighted.

```
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

akurasi = accuracy_score(y_test, y_pred)
presisi = precision_score(y_test, y_pred, average="weighted", zero_division=0)
recall = recall_score(y_test, y_pred, average="weighted", zero_division=0)
f1 = f1_score(y_test, y_pred, average="weighted", zero_division=0)

print(f"\nAkurasi   : {akurasi:.4f}")
print(f"Presisi   : {presisi:.4f}")
print(f"Recall    : {recall:.4f}")
print(f"F1-Score  : {f1:.4f}")

print("\nLaporan lengkap per kelas:")
print(classification_report(y_test, y_pred, zero_division=0))
```

Gambar 4.14 Kode tahap Evaluasi

Ringkasan nilai metrik yang diperoleh model ditunjukkan pada tabel berikut.

Metrik Evaluasi	Nilai
Akurasi	0,7363
Presisi (weighted)	0,7547

Metrik Evaluasi	Nilai
Recall (weighted)	0,7363
F1-Score (weighted)	0,7451

Berdasarkan tabel di atas, model Decision Tree memperoleh akurasi 73,63% dengan F1-Score weighted sebesar 0,7451. Model sangat baik mengenali kelas 0 (tidak diabetes), namun performanya rendah pada kelas 1 (pradiabetes) karena jumlah datanya yang sangat sedikit meskipun telah dilakukan SMOTE.

```
============================================================
8. EVALUASI
============================================================

Confusion Matrix:
[[31633   992  5387]
 [  666    30   230]
 [ 4530   275  2214]]

Akurasi   : 0.7371
Presisi   : 0.7541
Recall    : 0.7371
F1-Score  : 0.7453

Laporan lengkap per kelas:
              precision    recall  f1-score   support

         0.0       0.86      0.83      0.85     38012
         1.0       0.02      0.03      0.03       926
         2.0       0.28      0.32      0.30      7019

    accuracy                           0.74     45957
   macro avg       0.39      0.39      0.39     45957
weighted avg       0.75      0.74      0.75     45957
```

Gambar 4.15 Output kode Evaluasi

BAB V PENUTUP

5.1 Kesimpulan

- Program klasifikasi diabetes berhasil dibangun dan dijalankan sesuai alur, mulai dari input data hingga evaluasi.
- Tahap preprocessing menghapus 23.899 data duplikat dan menangani outlier, sedangkan SMOTE berhasil menyeimbangkan ketiga kelas pada data latih menjadi masing-masing 152.043 data.
- Model Decision Tree menghasilkan akurasi 73,63% dan F1-Score weighted 0,7451. Performa pada kelas minoritas masih rendah karena keterbatasan jumlah data.

5.2 Saran

- Mencoba algoritma lain seperti Random Forest atau XGBoost yang umumnya memberikan hasil lebih baik pada data tabular.
- Melakukan penyetelan parameter (hyperparameter tuning) pada Decision Tree, misalnya membatasi kedalaman pohon untuk mengurangi overfitting.
- Mempertimbangkan penggabungan kelas atau teknik resampling lain agar kelas pradiabetes lebih dikenali model.

DAFTAR PUSTAKA

Attari, M. Y. N., Ghadim, A. A., Ala, A., Simic, V., Tinka, D., & Pamucar, D. (2026). Application of healthcare data mining techniques to planning for nursing length of stay in surgical departments. *Systems and Soft Computing*, *9*, 200535. https://doi.org/10.1016/j.sasc.2026.200535

Olisah, C. C., Smith, L., & Smith, M. (2022). Diabetes mellitus prediction and diagnosis from a data preprocessing and machine learning perspective. *Computer Methods and Programs in Biomedicine*, *220*, 106773. https://doi.org/10.1016/j.cmpb.2022.106773

International Diabetes Federation. (2023, April 25). Indonesia. https://idf.org/our-network/regions-and-members/western-pacific/members/indonesia/

Alsulami, A., Khalifa, R., & Alghamdi, W. (2026). Improving performance and accuracy in decision trees: A literature survey on impurity functions. *International Journal of Advanced Computer Science and Applications (IJACSA), 17*(4), 984–995.

scikit-learn developers. (2026). *MinMaxScaler*. scikit-learn. https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.MinMaxScaler.html

Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research, 16*, 321–357. https://doi.org/10.1613/jair.953

Chicco, D., & Jurman, G. (2020). The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation. *BMC Genomics, 21*, 6. https://doi.org/10.1186/s12864-019-6413-7